

STAT/DATA 1810 · Notes

 Week 1 > Fnd 01 · Solution

Foundations 01 — Positron and the R Environment

STAT/DATA 1810 · Week 1 Wednesday · SOLUTION

AUTHOR

Instructor key

Name: Instructor key · **Date:** _____

Before anything else, type your name and today's date on the line above (and put your name in the **author:** line at the very top of this file).

How to read this document. Two labels show up over and over:

- **LM = Learning Moment.** I demonstrate; you follow along and run the code.
- **YT = Your Turn.** You do it yourself — fill the blank or the empty chunk.

Timing & labels

- **Budget:** panes + session 12' · running code 12' · objects 16' · functions/packages/help 20' · errors 12' · save & render 8' = **80'**
- **Checkpoint:** end of §2 at the **40' mark** — if behind, cut YT6 and YT7c
- **LM Learning Moment** — instructor demonstrates, students follow
- **YT Your Turn** — students do it themselves
- Say both names out loud the first time each appears — students otherwise guess

0 · Getting set up and what you are looking at 12 min

R is a programming language for working with data. **Positron** is the program we use to *talk to R* — an integrated development environment (IDE). R does the work; Positron is the workshop it happens in.

Setup checklist (do once, before class if you can)

Need bigger, step-by-step instructions? See [Prep — Getting Set Up](#).

Prep page

- Big-font, step-by-step version for the projector: [Prep — Getting Set Up](#) (also covers the Session tab and LM/YT)

Check each box as you finish it:

- ☐ **1. Install R, then Positron.** Download R from <https://cloud.r-project.org> and install it. Then download Positron from <https://positron.posit.co> and install it. (R first — Positron needs to find it.)
- ☐ **2. Download today's file.** From the course site, download the `.qmd` file for today. The file name has today's date and `qmd 1` in it — that is this document.
- ☐ **3. Build your folders on the Desktop.** On your Desktop, create:
 - a folder named `R`
 - a folder named `Data`
 - inside `R`, two folders: `Notes` and `Labs`

When you are done it looks like this:

```
Desktop/
├── R/
│   ├── Notes/
│   └── Labs/
└── Data/
```

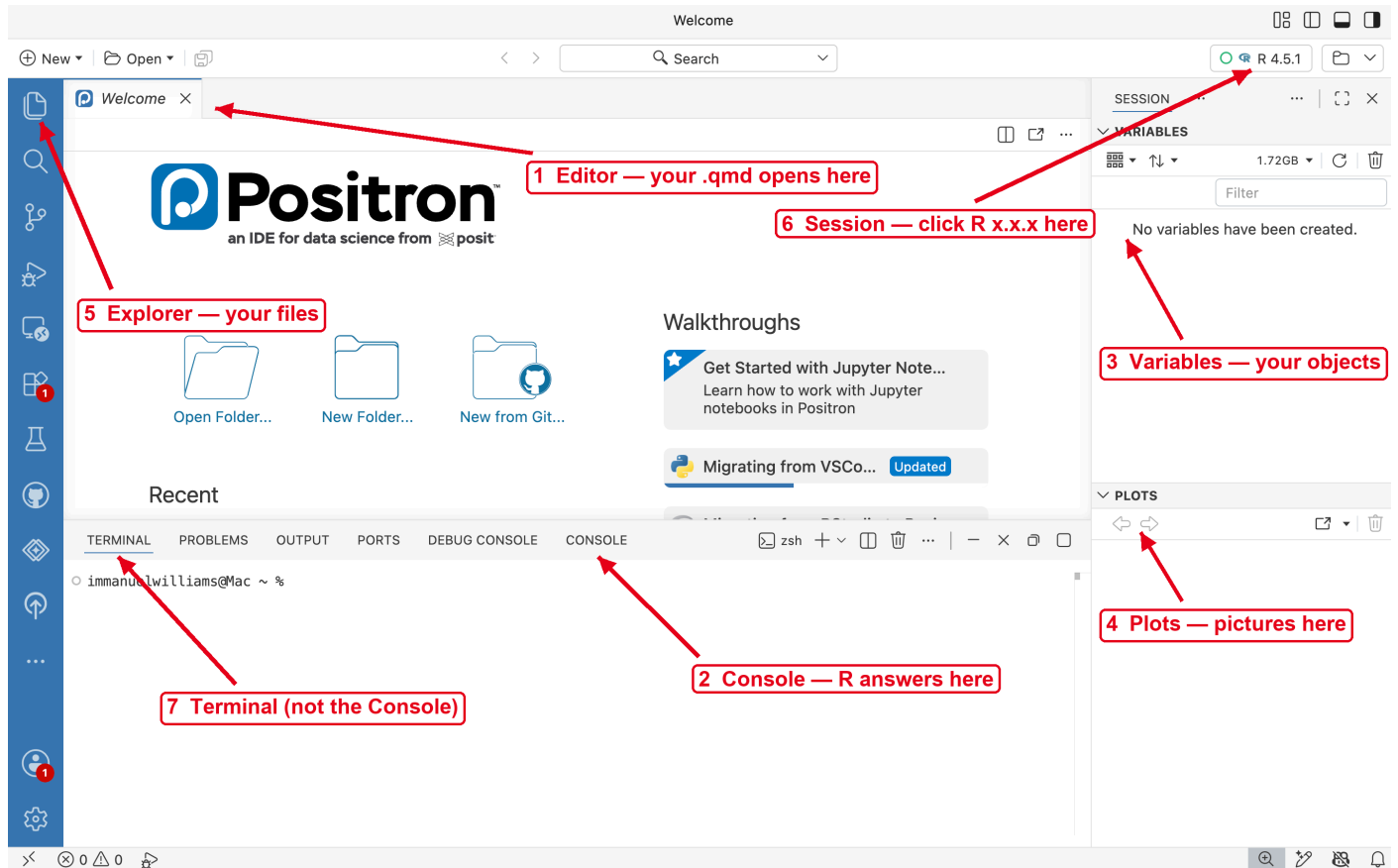
- ☐ **4. Move the file.** Drag the downloaded `.qmd` file out of `Downloads` and into `Desktop/R/Notes/`. (Notes go in `Notes`; labs will go in `Labs`; data files will go in `Data`. Every week.)
- ☐ **5. Open Positron.**
- ☐ **6. Open your folder.** In Positron: **File** → **Open Folder...** and choose `Desktop/R`. The folder's contents appear in the left sidebar.
- ☐ **7. Open the file.** In the left sidebar, click `Notes`, then click the `.qmd` file. It opens in the middle of the screen — you are now reading it inside Positron.
- ☐ **8. Fill in your name and date** at the top of this document.

Teaching note — setup

- The eight setup steps should be done **before** class: install R → install Positron → download the `.qmd` → make `Desktop/R/{Notes,Labs}` and `Desktop/Data` → move the file into `R/Notes` → open Positron → **File** → **Open Folder...** on `Desktop/R` → open the file from the sidebar → fill in name and date
- Budget **3 minutes** at the start to rescue whoever isn't there yet
- Most common failures:
 - the file is still in `Downloads`
 - they opened the *file* rather than the *folder*, so the Explorer sidebar is empty
- Fix for both: **File** → **Open Folder...** → `Desktop/R`
- Make them fill in name and date — first thing they type into a `.qmd`, and how you tell whose render is whose

The panes 6 min

Positron is divided into **panes**. The numbers in the picture match the list below — find each one on *your* screen and check it off. Physically point at it.



Positron's panes. Your screen will look like this, with your `.qmd` open in the Editor instead of the Welcome page.

- ☐ **1 • Editor** — the big middle area where this `.qmd` file is open. Code you *keep* lives here.
- ☐ **2 • Console** — bottom of the screen, the tab labeled **CONSOLE**. R answers here, one command at a time. Code you type here is *not* saved.
- ☐ **3 • Variables** — right side, under the **SESSION** tab. Every object you create this session shows up here. (Right now it says *No variables have been created* — remember what that looks like.)
- ☐ **4 • Plots** — right side, below Variables. Pictures appear here.
- ☐ **5 • Explorer** — the top icon in the left sidebar (two overlapping pages). Click it to see the files in the folder you opened.
- ☐ **6 • Session** — the `R 4.x.x` label in the top-right corner. More on this in a moment.
- ☐ **7 • Terminal** — the tab next to Console at the bottom. It is *not* R; it talks to your computer's operating system. Know it exists; we will not use it today.
- ☐ **Help** — opens on the right side when you ask R for documentation (section 3 today).

Teaching note — panes

- Put the annotated screenshot ([images/positron_panes.png](#)) on the projector
- Have students physically point at each numbered pane — the checklist **is** the activity
- Use Positron's own names consistently: **Editor**, **Console**, **Variables**, **Plots**, **Explorer**
- Two traps:
 - **CONSOLE** and **TERMINAL** tabs sit side by side — the Terminal shows a `%` prompt, not `>`
 - Variables/Plots live inside the **SESSION** panel on the right — if closed, the right side looks blank

- Terminal / Viewer / Connections get one sentence each — resist the tour becoming the lesson

YT1 (1 min). A neighbor asks “where do I look to see what objects I’ve made?” Which pane do you point at? **Variables**

YT2 (1 min). You type code and want it to be there tomorrow. Which pane does it go in? **The Editor** (this `.qmd`)

Answers

- YT1 Variables
- YT2 the Editor (this `.qmd`)

The Session 6 min

R runs inside a **session** — a running copy of R that Positron is talking to.

How to get to the Session tab. Look at the **top-right corner** of Positron (arrow **6** in the picture). It shows which R is running, something like `R 4.5.1`. That label is your session. Click it once: a small menu opens showing the running R, a **Restart** button, a **Shutdown** button, and any other interpreters installed on your computer. Close the menu (press Escape). Now look just below it on the right side: the panel labeled **SESSION** holds the **Variables** and **Plots** panes — everything the session is holding in memory, made visible.

Three things to know about the session, today:

1. **It has memory.** Every object you create lives in the session, and shows up in Variables.
2. **It can be restarted.** The Restart button (top-right session menu, or the Console toolbar) wipes the session’s memory clean — Variables empties out. Your *files* are untouched.
3. **It can be replaced.** The same menu lets you pick a different R (or Python) if more than one is installed. Don’t do that today; just know the button exists.

Do this now: create nothing yet, look at Variables (empty), and remember what empty looks like — we’ll come back after we’ve made some objects.

Teaching note — getting to the Session tab

- Point at the top-right interpreter label (`R 4.x.x`, arrow 6 on the screenshot)
- Click it once so they see the menu: running session, Restart, Shutdown, other interpreters — then press Escape
- Point at the **SESSION** panel beneath it: “that label is the session; this panel is what the session remembers”
- Say the three facts — **memory / restartable / replaceable** — and move on
- Payoff comes in §4 LM11 when they restart for real
- Show that Variables is empty *now* — the “before” picture makes the restart in §4 land

Q1. In one sentence: what is the difference between the Console and the Editor?

Your answer: The Console runs code once and forgets it; the Editor holds code you keep, so you can re-run everything tomorrow.

Q1 model answer

- **Q1** The Console runs code once and forgets it
- The Editor holds code you keep, so you can re-run everything tomorrow

1 · Running code and code chunks 12 min

LM1 — the Console (2 min). Click into the Console, type `2 + 2`, press Enter. R answers. Now press the up-arrow — the Console remembers your last line. Edit it to `2 + 3` and press Enter.

LM2 — a code chunk (5 min). This document is a **Quarto** file: text and code mixed together. The gray boxes are **code chunks**. A chunk is:

- opened by a line that reads ```{r}` (three backticks, then `{r}`)
- closed by a line of three backticks ````
- everything between the two is R code

Look at the chunk below in the Editor. Notice the small **play button** at its top-right corner. Click it (or put your cursor on a line and press **Cmd/Ctrl + Enter** to run just that line):

```
# LM2: this line is a COMMENT – R ignores anything after #
2 + 2
```

```
[1] 4
```

```
3 * 7
```

```
[1] 21
```

```
10 / 4
```

```
[1] 2.5
```

Two things happened: the results appeared **under the chunk** and the same code ran in the **Console**. A chunk is just a saved bundle of Console lines.

LM3 — chunk options (2 min). Lines that start with `#|` at the top of a chunk are **options** — instructions to Quarto, not R. You will see `#| eval: false` in this document; it means “show this code, but don’t run it when rendering.” Useful for code that is broken on purpose.

```
# LM3: this chunk is displayed but never runs
this_would_be_an_error
```

Teaching notes — LM1-LM3

- **LM1** The up-arrow trick saves students dozens of retypes this term — worth the 20 seconds
- **LM2 Anatomy of a chunk:**
 - put the cursor on the ```{r}` line and name its parts out loud: fence, `{r}` = “this is R”, fence
 - click the play button so they see output land **both** under the chunk and in the Console
 - the sentence to leave them with: “a chunk is a saved bundle of Console lines”
 - students create their own chunks starting Monday (Foundations 02) — today they only run and edit ours
- **LM3** One sentence on `#|` options is enough — they’ll write `#| label:` and `#| eval: false` themselves in ETV cycles

YT3 (3 min). Edit the chunk below so it computes *THREE* things: 5 minus something to give 1; 100 divided by 8; and 2 to the 10th power (hint: `^`).

```
# YT3: edit and run – start with: 5 - ____
5 - 4
```

```
[1] 1
```

```
100 / 8
```

```
[1] 12.5
```

```
2^10
```

```
[1] 1024
```

Q2. Where did the output show up — under the chunk, in the Console, or both? **Both**

Expected — YT3

- **YT3** `1 · 12.5 · 1024`
- **Q2** **both** (output appears under the chunk *and* in the Console)

2 · Objects and assignment 16 min

LM4 — the arrow (3 min). `<-` stores a value under a name. Without it, R computes and forgets. (Type it fast: `Alt + -` inserts `<-` in Positron.)

```
# LM4: create an object, then call its name
cups <- 3
cups
```

```
[1] 3
```

Look at the **Variables pane** — `cups` is there now, with its value.

LM5 — objects combine (4 min). Predict before you run: what will `total` be?

```
# LM5: predict, then run
price <- 4.50
total <- cups * price
total
```

```
[1] 13.5
```

Q3. Prediction: **13.5** · What R said: **13.5**

LM6 — overwriting (4 min). Run this, then check the Variables pane again.

```
# LM6: same name, new value
cups <- 10
total
```

```
[1] 13.5
```

Q4. `total` did NOT change to 45. Why not? (Hint: WHEN was `total` computed?)

Your answer: `total` was computed at the moment the `<-` line ran, using `cups = 3`. Changing `cups` later doesn't re-run old lines — R objects store values, not formulas.

Q4 model answer — the most important idea of the day

- Q4 `total` was computed at the moment the `<-` line ran, using `cups = 3`
- Changing `cups` later doesn't re-run old lines — **R objects store values, not formulas**
- Spend the full minutes here
- Spreadsheet contrast lands well: Excel cells recompute; R objects don't

YT4 (5 min). In the chunk below: (a) create `hours` holding how many hours you slept last night; (b) create `minutes` as `hours` times 60; (c) print `minutes`; (d) predict — if you now change `hours` to 8, does `minutes` change? Test it.

```
# YT4: any sleep value; e.g.
hours <- 7
minutes <- hours * 60
minutes
```

```
[1] 420
```

```
hours    <- 8
minutes      # still 420 – same lesson as Q4, now self-discovered
```

```
[1] 420
```

Your prediction for (d): **No** — *minutes* stays 420 · What happened: **It stayed 420**

YT4 (d) answer

- YT4 *minutes* does **not** change — Q4 again, but the student built it, so it sticks
- Have two students say their prediction and result aloud
- Then point at Variables: *cups*, *price*, *total*, *hours*, *minutes* — five objects, all session memory

Now look at Variables: how many objects are there? Names are on the left, values on the right — this pane is your session's memory, made visible.

3 · Functions, packages, help 20 min

LM7 — a function (4 min). A function is a named action: `name(arguments)`. *mtcars* is a dataset built into R — 32 cars from 1974. No import needed. The `$` pulls one column out of it (much more on `$` next week).

```
# LM7: call functions on built-in data
mean(mtcars$mpg)
```

```
[1] 20.09062
```

```
nrow(mtcars)
```

```
[1] 32
```

Expected — LM7

- LM7 20.09062 · 32

YT5 (4 min). One line each: (a) `max()` of horsepower, `mtcars$hp`; (b) `min()` of miles-per-gallon; (c) `ncol()` of *mtcars* — how many columns? (d) `round()` the mean mpg to 1 decimal place. (Hint: `round(x, 1)`.)

```
# YT5:
max(mtcars$hp)
```

```
[1] 335
```

```
min(mtcars$mpg)
```

```
[1] 10.4
```

```
ncol(mtcars)
```

```
[1] 11
```

```
round(mean(mtcars$mpg), 1)
```

```
[1] 20.1
```

Expected — YT5

- YT5 335 (the Maserati Bora) · 10.4 · 11 · 20.1
- Part (d) is the first *nested* call — say “inside out” once; don’t dwell

LM8 — help (5 min). Every function has documentation. Run this, and the Help pane opens:

```
?mean
```

Read the **Usage** and **Arguments** sections. That’s where the answers to “what does this function want from me?” always live.

Q5. From the help page: what does the *na.rm* argument do?

Your answer: *na.rm = TRUE* tells *mean()* to remove *NA* values before computing; the default *FALSE* returns *NA* if any value is missing.

YT6 (2 min). Look up *?round*. What is the name of its second argument, and what is its default value? *digits · 0*

LM9 — packages (5 min). A **package** is a collection of functions someone wrote. You **install** once (download to your computer), you **load** every session (open the toolbox):

```
install.packages("tidyverse") # ONCE per computer — done in setup
library(tidyverse)           # EVERY session you use it
```

```
# LM9: load it now (already installed)
library(tidyverse)
```

Q6. Fill the blanks: *install.packages()* runs **once** per computer; *library()* runs **once** per session.

Q5 · YT6 · Q6 answers

- **Q5** `na.rm = TRUE` tells `mean()` to remove `NA` values before computing; the default `FALSE` returns `NA` if any value is missing
 - plant the flag: this argument returns in Foundations 03 and every ETV cycle
- **YT6** `digits`, default `0` — the purpose is reading a Usage line, not the fact itself
- **Q6** install runs **once per computer**; library runs **once every session**
 - analogy that sticks: install = buying the drill; library = taking it out of the garage

4 · When things break 12 min

LM10 — read the error (4 min). Run this broken line. Read the message *slowly* before fixing it.

```
# LM10: broken on purpose – fix it in the chunk below
meen(mtcars$mpg)
#> Error in meen(mtcars$mpg) : could not find function "meen"
```

```
# LM10 fixed:
mean(mtcars$mpg)
```

```
[1] 20.09062
```

Q7. What did the error message say, and what did it mean?

Your answer: could not find function "meen" — the name doesn't exist: a typo, or a package not loaded.

Q7 model answer

- **Q7** “could not find function” means the *name* doesn't exist — a typo, or a package not loaded
- Name the two most common error families now:
 - `could not find function` → typo / `library()` missing
 - `object not found` → name typo / never created
- Students who can *classify* an error stop fearing errors

YT7 (4 min). Each chunk below has ONE bug. Read the error, then fix it in place. Write the error family in the comment (typo in a function name? object that doesn't exist? something else?).

```
# YT7a – error family: ____
mean(mtcars$mpg)
```

```
# YT7a – error family: column-name typo (`$` on a missing column gives NULL, not an
  error)
mean(mtcars$mpg)
```

```
[1] 20.09062
```

```
# YT7b – error family: ____
mean(Mtcars$mpg)
```

```
# YT7b – error family: object not found (R is case-sensitive)
mean(mtcars$mpg)
```

```
[1] 20.09062
```

```
# YT7c – error family: ____
mean(mtcars$mpg
```

```
# YT7c – error family: unbalanced parentheses (missing `)`
mean(mtcars$mpg)
```

```
[1] 20.09062
```

YT7 answers — three error families

- **YT7a** `mtcars$mpg` → returns `NULL`; `mean` warns “argument is not numeric or logical: returning NA”
 - family: **column-name typo** — `$` on a missing column gives `NULL`, not an error (the sneakiest of the three)
 - fix: `mpg`
- **YT7b** `Mtcars` → `Error: object 'Mtcars' not found`
 - family: **object not found** — R is case-sensitive
 - fix: `mtcars`
- **YT7c** `missing )` → Console shows a `+` prompt waiting for more; in a chunk, “unexpected end of input”
 - family: **unbalanced parentheses**
 - fix: add `)` — and show them **Escape** in the Console to get out of a `+` prompt

LM11 — the reset button (4 min). If R gets confused, restart the session: Console toolbar → **Restart R** (or the top-right session menu). Do it now, for real. Then look at Variables.

Q8. After the restart: is `cups` still in Variables? **No** · Is the code that **CREATED** `cups` still in this file? **Yes** · Re-run section 2’s chunks — how long did it take to get `cups` back? **About 10 seconds**

LM11 — restart for real

- **LM11** Everyone restarts → Variables empties
- **Q8** `cups` gone · code still here · re-running §2 takes ~10 seconds
- That 10-second recovery is the argument for putting code in the file

5 · Save & render 8 min

Save this file into your project's `R/` folder as `fnd01_yourlastname.qmd`, then press **Render** (the button above the Editor, or Cmd/Ctrl + Shift + K). A webpage appears — your first reproducible document. Every chunk in it re-ran from a *fresh* session to build that page.

Exit question: name one thing that lives in the Variables pane and one thing that lives in the Editor, and say which one survives a restart.

Your answer: `cups` lives in Variables and is gone after a restart; this `.qmd` (and the code that recreates `cups`) lives in the Editor and survives.

Exit model answer

- `cups` lives in Variables → gone after restart
- this `.qmd` lives in the Editor → survives
- the code that *recreates* `cups` survives — the entire argument for reproducible documents, made on day one without ever saying “CLO 9”